

AI-Ready-Repo Engine

Making Any Codebase Agent-Ready

SwarmAI Delivery Engine • v1.0 Draft • 2026-05-29 • Author: XG + Swarm

Transforms any codebase into DDD-structured artifacts (PRODUCT.md, TECH.md, IMPROVEMENT.md, PROJECT.md) that make AI agents truly understand a project — not just navigate code, but comprehend purpose, architecture, history, and current state.

Output targets: Kiro IDE, Claude Code, Codex, and future agent-powered IDEs.

Positioning



Internal: DDD powers pipeline judgment | External: AI-Ready-Repo exports DDD as universal standard

Narrative: "AI-Ready-Repo Standard uses DDD (Domain-Driven Documentation) to give any AI agent complete project understanding — why it exists, how it works, what failed before, and what matters now."

Problem

AI coding agents face a **cold-start problem** on every existing codebase. They can parse syntax but cannot read your team's mind.

Evidence of Demand

Signal	Source	Date
"Long-term memory" is top community question	#kiro-interest	2026-05-28
Brownfield Bootstrapper: 15 reactions in 24h	#amazon-builder-genai-power-users	2026-05-27
MeshClaw sessions start at ~38% context consumed	#meshclaw-interest	2026-05-28
Community memory solutions filling gap (KiroMem, KiRoom)	#kiro-interest	2026-05-28
Claude Code plugins: 333+ tokens each, always active	#claude-code-internal-interest	2026-05-28

Gap in Current Solutions

Solution	Covers	Misses
CLAUDE.md	Build commands, basic rules	Architecture, history, priorities, non-goals
agents.md spec	Template for navigation	No generation, no refresh, flat file
Brownfield Bootstrapper	AGENTS.md + specs (Amazon-only)	No biz context, no self-maintenance
Kiro steering docs	User-written rules	No auto-generation from codebase

Key Insight: Why DDD, Not a Flat File

A single AGENTS.md (≤150 lines) can tell an agent *where things are*. It cannot teach judgment.

PRODUCT.md → "We don't do caching because regulatory requires real-time"
TECH.md → "All DB access through repository.ts, never raw SQL"
IMPROVEMENT.md → "We tried event sourcing in Q1, reverted after 3 incidents"
PROJECT.md → "Currently migrating auth – don't touch identity module"

Same 4-file structure used across 8 active projects. Battle-tested over 3 months with automated cultivation.

User Scenarios

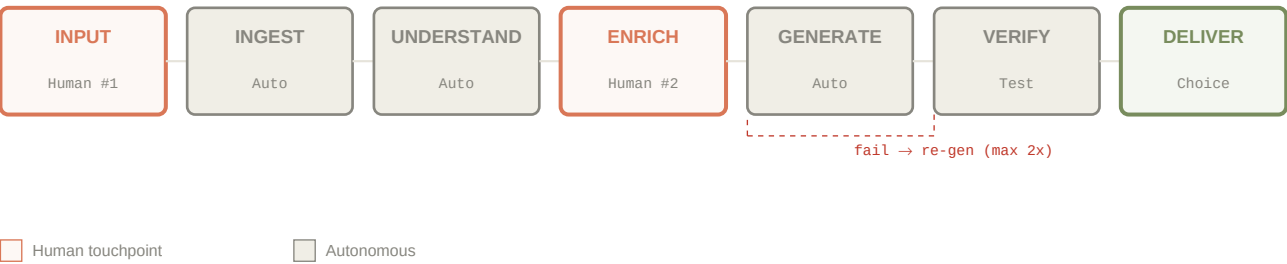
Role	Scenario	Before → After	Touches
Engineer	"Make my team's service AI-ready"	Agent tried to add direct DB calls violating repository patterns. After AI-Ready: PROBLEM SOLVED	Reviews AI-Ready: agent
PM / TPM	"Agent must understand business context"	Agent proposed caching exchange rates (regulatory violation). After AI-Ready: agent understood	Reviews AI-Ready: agent
Knowledge Expert	"Capture tribal knowledge before people leave"	People leave. After AI-Ready: 15 evidence-grounded questions. After AI-Ready: 15 evidence-grounded questions	Reviews AI-Ready: agent
Director	"AI-readiness across the org"	No visibility into why some teams' agents fail. After AI-Ready: aggregated scorecard, only investment	Reviews AI-Ready: agent
Cross-team Handoff	"New team inheriting a service"	80-page stale Confluence. After AI-Ready: install.sh - Agents install the devops. After AI-Ready: install.sh	Reviews AI-Ready: agent

3 Levels of AI-Readiness

Level	What's Present	Agent Can	Agent Cannot
1: Navigable	AGENTS.md + TECH.md + code-intel.json	Find files, run build, follow conventions	Make judgments, avoid gotchas
2: Safe	+ IMPROVEMENT.md	+ Avoid pitfalls, understand blast radius	Make business decisions
3: Autonomous	+ PRODUCT.md + PROJECT.md (full DDD)	Judgment calls, boundaries, priorities	Nothing — full understanding

Engine always targets Level 3. Gracefully degrades to Level 2 if user skips business context.

E2E Flow (7 Phases)



- Phase 1 INPUT:** Repo path(s) required. Optional: docs, wiki, Slack, issues, runbooks, existing DDD.
- Phase 2 INGEST:** Clone repos, parse docs, detect tech stack (LLM-driven, all languages Day 1).
- Phase 3 UNDERSTAND:** Build code graph, detect modules, mine tribal knowledge from git (WHEN→RISK→BECAUSE grammar).
- Phase 4 ENRICH:** Max 5 targeted questions (all optional, skippable).
- Phase 5 GENERATE:** Produce all DDD files + AGENTS.md + code-intel.json + REVIEW-REPORT.md.
- Phase 6 VERIFY:** Fresh agent self-test (3 tasks from real git history). Fail → re-gen max 2x.
- Phase 7 DELIVER:** Install to workspace / Push to Git / Zip bundle / Raw files.

Output Artifacts

← Entry point (≤ 150 lines)

.ai-ready/

PRODUCT.md

← Purpose, non-goals, constraints

TECH.md

← Architecture, conventions, invariants

IMPROVEMENT.md

← Failures, gotchas, patterns

PROJECT.md

← Priorities, decisions, blockers

code-intel.json

← Modules, deps, blast radius

ai-ready.json

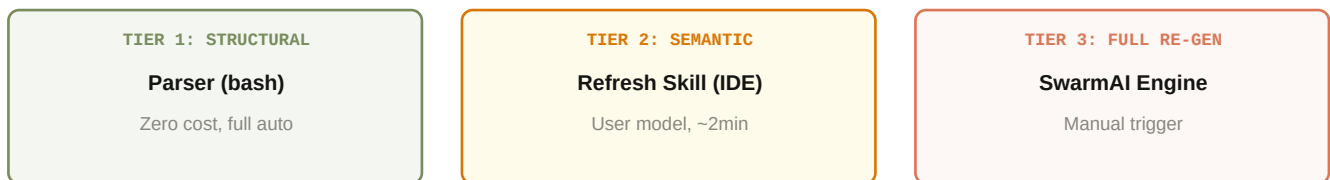
REVIEW-REPORT.md

← Confidence, gaps, review assignments

AGENTS.md = entry point (≤150 lines, links to .ai-ready/). DDD files provide depth. code-intel.json provides queryable structure. REVIEW-REPORT.md guides human review.

Refresh Mechanism — Self-Maintaining Artifacts

Problem: Static artifacts become actively misleading as code evolves. Stale context is worse than no context.



Cadence: Daily → T1 silent | Weekly → occasional T2 | Quarterly → T3

Tier	Trigger	Cost	Scope
1: Structural	Hook on file/config change	Zero (bash)	code-intel.json only
2: Semantic	User says "refresh ai-ready"	User's model (~2 min)	AGENTS.md + DDD sections
3: Full Re-gen	Major refactor / quarterly	SwarmAI engine session	All artifacts

Decay markers: [■■ unverified 14d] inline — agent treats as "possibly stale, verify before relying."

Knowledge Ingestion — User → Artifacts

Refresh handles Code→Artifacts. But there's another direction: **User provides knowledge** → **Artifacts enriched**. Without this, corrections and decisions are lost between sessions.

Three Trigger Paths

Trigger	Example	Target
1. Explicit statement	"ai-ready learn: never deploy on Fridays"	IMPROVEMENT.md
2. Document ingest	User drops PRD → "ai-ready ingest design.md"	PRODUCT.md + TECH.md
3. Correction capture	Agent corrected → "Want to capture this?"	TECH.md or IMPROVEMENT.md

Classification Logic

Input Signal	Target File	Keywords
Purpose, constraints, audience	PRODUCT.md	"out of scope", "compliance", "we don't do"
Conventions, architecture, patterns	TECH.md	"always use", "never call", "pattern"
Failures, gotchas, incidents	IMPROVEMENT.md	"burned by", "broke when", "revert"
Priorities, blockers, decisions	PROJECT.md	"this quarter", "blocked", "decided to"

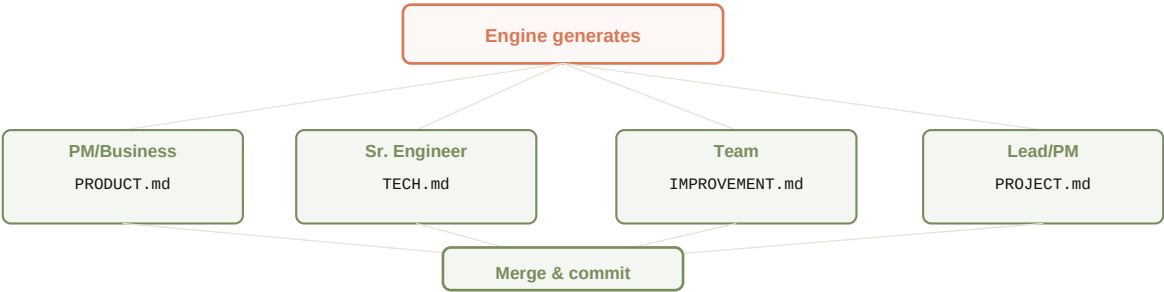
Rules

- NEVER overwrite existing entries (append only)
- ALWAYS cite source: [added: date, source: user/filename]
- ALWAYS ask confirmation before writing
- If unclear → ask user which file

Full Lifecycle

Day 1: Engine generates → install → baseline context
Day 3: Code changes → Tier 1+2 refresh
Day 5: User drops PRD → ingest → PRODUCT.md enriched
Day 8: Agent corrected → capture → TECH.md gains convention
Day 12: User "learn: deprecated v1 API" → PROJECT.md updated
Day 30: 50+ commits → Tier 2 refresh (preserves learned entries)
Day 90: SwarmAI Tier 3 re-gen (all learned entries as baseline)

Multi-Stakeholder Review Flow



File	Reviewer	Review Focus
PRODUCT.md	PM + Business stakeholder	Goals, boundaries, constraints accurate?
TECH.md	Senior Engineer / Architect	Conventions, architecture match reality?
IMPROVEMENT.md	Whole team	Gotchas still current? Anything missing?
PROJECT.md	Lead + PM	Priorities, blockers, decisions up to date?

AI-Ready Score (8 Dimensions)

Dimension	0 (unusable)	5 (passable)	10 (excellent)
Navigation	No README	README stale	AGENTS.md + map + entry points
Build/Test	Can't run	Commands fail	Verified one-command build+test
Architecture	Black box	README ≠ code	TECH.md matches reality
Conventions	No rules	In comments	Structured, prescriptive
Tribal Knowledge	In people's heads	In commit msgs	IMPROVEMENT.md evidence-grounded
Code Graph	None	Partial	Full deps + blast radius
Test Safety	No tests	Broken CI	CI + coverage + safety paths
Ops Context	Can't deploy	Runbook somewhere	Deploy chain + dashboards

Installation — Zero Config

One command: `bash install.sh` — auto-detects IDE (Kiro vs Claude Code), places files, installs refresh hook.

Conflict Resolution

Situation	Behavior
AGENTS.md exists	SKIP. Suggest merge. Never overwrite.
CLAUDE.md exists	Don't touch. Claude Code reads both.
.ai-ready/ exists	Incremental update. Preserve <!-- user --> sections.
settings.json has hooks	Merge ours (tagged _source for clean uninstall).

Competitive Comparison

Dimension	Bootstrapper	agents.md	AI-Ready-Repo (ours)
Knowledge model	Flat AGENTS.md	Template	DDD 4-file (layered)
Scope	Code nav only	Code nav	Full understanding (why+how+history+now)
Stakeholders	Engineer only	Engineer	PM + Engineer + Team + Director
Self-maintenance	None (static)	N/A	3-tier refresh + decay
Platform	Amazon-only	Agnostic (spec)	Agnostic (spec + engine)
Verification	None	None	Fresh agent self-test
Install UX	Manual copy	Manual	Zero-config auto-detect

Where Others Are Better

- **Bootstrapper:** Deeper Amazon integration (auto-mines CRs, SIM, wiki)
- **Kiro native:** Tighter IDE integration (no install.sh if Kiro builds DDD in)
- **agents.md:** Simpler adoption curve (1 file easier than 4+1)

Open Standard Strategy

What We Publish (open)

- Spec: DDD structure for AI-ready codebases
- Scoring rubric: 8-dimension self-assessment
- Template files + philosophy doc

What We Keep (SwarmAI advantage)

- The engine (auto-generation from code + signals)
- Refresh mechanism (self-maintaining artifacts)
- Verify phase + multi-package synthesis

Narrative: "The standard is open. The best way to achieve it is SwarmAI."

Security & Limitations

Security

- Engine runs locally. No code leaves device unless user pushes to git.
- Artifacts contain structure metadata, NOT source code content.
- install.sh: pure local file ops. No network, no telemetry.

Known Limitations

Limitation	Mitigation
Monorepos >1M LOC	Sampling: top 200 files by churn + all entry points
PROJECT.md half-life (~2-4 weeks)	Freshness marks stale; designed for human update
Tier 2 model dependency	Mechanical steps; falls back to "recommend re-gen"
AGENTS.md auto-read (unverified)	Must verify; fallback: add ref to CLAUDE.md

Design Principles

#	Principle	Implementation
1	Detect, don't assume	Auto-detect stack from code. LLM-driven, all languages Day 1.
2	≤150 lines entry point	AGENTS.md is brief; DDD docs go deeper, loaded on demand.
3	Every line earns its place	"Will agent make systemic mistake without this line?"
4	Two human touchpoints	Input + Enrich. Everything else autonomous.
5	Zero-config installation	Auto-detect IDE, one command, never overwrite.
6	Self-maintaining artifacts	Refresh skill + parser. IDE agent maintains them.
7	Knowledge has layers	Different stakeholders review different docs.
8	Knowledge must evolve or die	Freshness tracking, decay markers, staleness detection.
9	Judgment > Description	"Never call X" beats "X exists".
10	Evidence-grounded	Backed by commit/issue. Can't ground = don't write.

Implementation Milestones

M#	Deliverable	Validation
M1	Single-repo DDD generation (all languages)	Manual review on real cold-start project
M2	Self-maintaining (parser + skill + hook)	Add module → parser fires → notification
M3	Verified output (VERIFY phase)	Omit module → VERIFY fails → re-gen succeeds
M4	Multi-package (fan-out + cross-system)	3-repo system with cross-deps
M5	IDE adapters (Claude Code + Kiro)	E2E: install → task → agent succeeds
M6	Published standard + spec	Community engagement (stars, forks)

Estimated: ~7 sessions to full capability.

Success Criteria

- Developer runs install.sh → agent correctly navigates codebase on first task
- PM reviews PRODUCT.md and says "yes, this captures our intent"
- After 50 commits, Tier 1 detects staleness, Tier 2 refreshes in <2 min
- Multi-package (3 repos) → each gets DDD + cross-system awareness
- Uninstall removes all files + hooks cleanly, zero residue
- Published spec gets community adoption
- AI-Ready Score improves ≥4 points from baseline